

NORWEGIAN UNIVERSITY OF LIFE SCIENCES
(NMBU)

REPORT: SIMULATION OF AN OIL SPILL

WILL THE OIL SPILL AFFECT THE
FISHING GROUND?

AUTHORS

INGRID VESTVIK
ELIAS SIGURD KROKEN
RAJVIR SINGH AUJLA

NORWAY, JANUARY 2025

Contents

Contents	i
1 Task Description	1
1.1 Formulas	1
2 User Guide	3
3 Code Structure	4
4 Agile Development	5
5 Results	7
6 Reflections	8

1 Task Description

In this report, three students have analyzed the impact of an oil spill on the fishing town of "Bay City". The task was to determine whether the oil spill would spread to the fishing grounds or not. This was achieved by dividing the map into triangles, called Mesh cells and also by using available data on the ocean's flow field, the initial location of the oil spill, and other mathematical models, mentioned in the **Formulas** chapter.

The result of this analysis is a simulation that predicts the movement of the oil spill based on the underlying flow field. The simulation is represented in a series of several images, each showing the position of the oil spill over time. The images are color-coded using the "viridis" color scale, which maps the amount of oil in each cell: the more oil present, the more yellow the cell appears. The complete sequence of images is compiled into a video, illustrating the progression of the oil spill from start to finish.

1.1 Formulas

To simulate this oil movement, the area have been divided into small triangle shaped cells. This is called a mesh and a sketch of one triangle cell can be found under the formula table. By calculating the outward pointing normals to each cells it is possible to combine these normals with the flow field and by this calculate the oil movement. Here is a list of the different variables and the most important formulas used in the simulation.

Variable overview

Variable	Name
u	oil value
v	flowfield
normal	normal vector
i	triangle cell index
ngh	neighbor cell index
e	facet of the triangle
A_i	area of a triangle
F	flux value
up	amount of oil update in each cell

Testing formulas inside a table: Tables should be used to for example define simulation parameters. Include tables in the following way:

Name	Formula
Initial Oil	$u(t = 0, \vec{x}) = e^{-\frac{(\ \vec{x} - \vec{x}_t\)^2}{0.01}}$
Initial Coordinate	$\vec{x} = (0.35, 0.45)$
Flowfield	$v(\vec{x}) = \begin{pmatrix} y - 0.2x \\ -x \end{pmatrix}$
Flux	$F_i^{(ngh,i)} = \frac{-\delta t}{A_i} \cdot g(u_i^n, u_{ngh}^n, \text{normal}, \frac{1}{2}(\vec{v}_i + \vec{v}_{ngh}))$
Update function	$up = \begin{cases} a \cdot (\vec{v}, \text{normal}), & \text{if } \vec{v} \cdot \text{normal} > 0 \\ b \cdot (\vec{v}, \text{normal}), & \text{otherwise} \end{cases}$
Oil amount at time t	$u_i^{(n+1)} = u_i^n + F_i^{(ngh_1,n)} + F_i^{(ngh_2,n)} + F_i^{(ngh_3,n)}$

A visualization of one triangle cell in the Mesh. From this triangle the oil is calculated by the outward pointing normals.

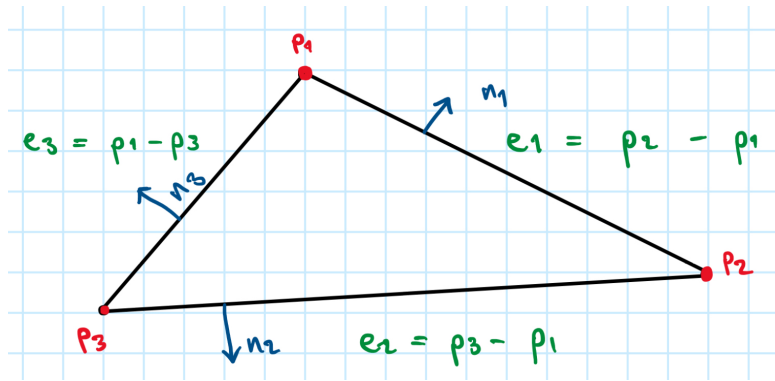
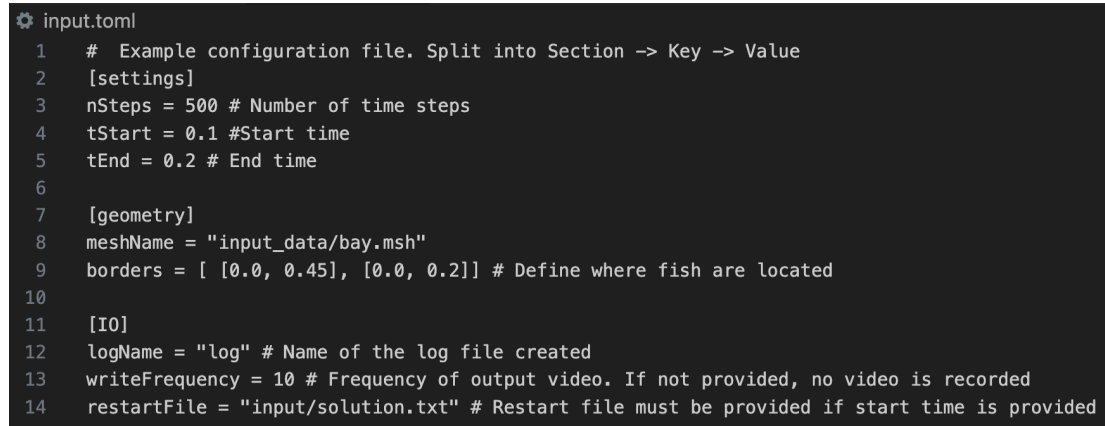


Figure 1.1: Triangle Cell

2 User Guide

Ease of use for the user has been prioritized. The configuration is handled through a toml file, allowing users to customize settings without modifying the main code. All necessary adjustments can be made in the input.toml file, as illustrated in Figure 2.1.



```
input.toml
1  # Example configuration file. Split into Section -> Key -> Value
2  [settings]
3  nSteps = 500 # Number of time steps
4  tStart = 0.1 #Start time
5  tEnd = 0.2 # End time
6
7  [geometry]
8  meshName = "input_data/bay.msh"
9  borders = [ [0.0, 0.45], [0.0, 0.2] ] # Define where fish are located
10
11  [IO]
12  logName = "log" # Name of the log file created
13  writeFrequency = 10 # Frequency of output video. If not provided, no video is recorded
14  restartFile = "input/solution.txt" # Restart file must be provided if start time is provided
```

Figure 2.1: User guide for toml

In this file, users can change parameters such as the number of time steps, start time, and end time. These settings will automatically update the time step size (delta-t) used in the main code for the simulation.

To determine which toml files to use the user also has the option to write commands in the terminal. There's three options:

- To process a single config file: `main.py -c example.toml`
- Process all .toml files in the current folder: `main.py -find_all`
- Process all .toml files in a specified folder: `main.py -find_all -f ./configs`

Once the settings have been configured, the user can run the main file to start the simulation. By doing this, the simulation will start to generate an oil distribution plot from the start coordinates $x, y = 0.35, 0.45$, which will be saved in the images folder for further analyses.

3 Code Structure

The code is built up by classes that are structured in suited files and folders. The codebase is organized into several main components described under. This structure allows for separation of concerns and makes it straightforward to add new features, test individual components, and manage dependencies.

The input toml is the file where the user can change settings as described in the [User Guide](#) chapter.

The main file works like the entry point to the program. It imports classes and functions from the packages folder and executes the program logic.

The "packages" folder is the core of our project, consisting of four files: msh-classes, simulation, logger and plot-animation.

The msh-classes defines the fundamental classes used in the program, such as Point, Cell, Triangle, Line and Mesh. These classes handle data related to mesh structure, including the storage and management of points, cells, and the relationships between neighboring cells.

The simulation file inherits the classes defined in the msh-classes file, adding additional functionality for simulation purposes. It includes various formulas and methods for computing flux, area, midpoints and other calculations necessary for the simulation. By separate these two files, we ensure expandability and a modular program. The simulation file also reads the result into a restart-file. This will make it possible for the user to end and restart the code at a chosen time.

The logger file sets up a logging system to record simulation events. It includes functions to initialize the logger, log simulation parameters (like time steps, mesh, and area), and track the amount of oil in the fishing grounds area over time. It stores the logs in a specified file, capturing key information for monitoring or debugging the simulation. The code is also printing out which processes that have been done at different time for debugging.

The last file in packages is plot-animation. This file visualize the simulation results. It consists of a plot function that generates the oil spill over time. These plots are saved both as individual images at each time step and as a video that compiles the entire simulation.

The input_data folder contains the bay-msh file which is the map we are simulating on. The input folder stores the visual output of the simulation, including individual images and video files generated by the plot-animation module.

The tests folder contains test files matching to each package file. We chose to test the most critical parts of the project. We would preferably have liked to make tests for all functions but we did not have enough time.

The restart file is a CSV file that stores all chosen mesh information at each time step. We chose CSV format over text files because it offers better data compression and facilitates easier reading and writing operations.

4 Agile Development

Throughout the project, the group maintained strong communication and organization. The group used inspiration from the Scrum technique both in the way to keep track on each task and also to delegate the tasks. In the initial stage, the group members took time to discuss the task, draw the problem and a possible structure and also get to know each group members expertize. This early conversation was valuable as it allowed us to delegate tasks effectively and set a clear direction for the project. From a scrum perspective we all had a role as developers but we did have different responsibilities.

To stay organized and maintain a good structure we created a Product Backlog and Sprint Backlog on GitHub. The Product Backlog outlined all the tasks that needed to be completed throughout the project, while the Sprint Backlog focused on what we planned to accomplish in the short term. To emphasize the importance of specific tasks we used a MVP-label, minimum viable product, to mark the items that were essential to the project. This structure helped us prioritize tasks and ensure we were focusing on the most critical work in the initial stage.

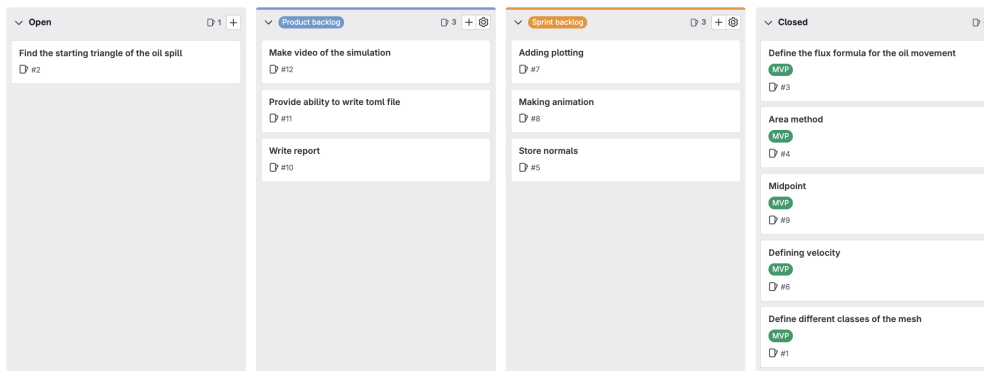


Figure 4.1: *GitBoard in the middle of our project*

In addition to the backlogs we created to-do lists in OneNote for each group-member. We held daily check-ins where we shared updates on what had been completed and discussed any difficulties. An example of our to-do list shown under. This open communication kept everyone aligned and allowed us to address issues as they rose. For instance our original plan was to complete a working simulation by Friday 9th of January but because of different issues a working simulation was not finalized before 17th of Januray. While this caused some stress, our ability to adapt and communicate effectively kept us on track. We agreed that one of the group members should continue the work on making the TOML files while two of the members could work with resolving the technical issue. This allowed the group to continue making progress even if the debugging took a bit longer than expected.

- ☐ Fishing area
 - ☒ Add in plot
 - ☒ Midpoint of cell is inside area
 - ☒ Sum of u in each time pic is taken
 - ☒ Plot the u graph
 - ☐ Find the max value for summary
- ☐ Move to innit
 - ☐ Area calc
 - ☐ V calc
 - ☐ Neighbor calc
 - ☐ Normal calc
 - ☐ Midpoint calc
- ☐ Neighbor calc
 - ☒ Add break
 - ☐ Append also to neighbor cell
- ☒ Add line to updating oil (so oil can disappear)
- ☐ Add a read and write function to a txt file
 - ☒ Write
 - ☐ Read
- ☐ Make tests
 - ☒ Point class
 - ☒ initialization
 - ☒ Repr
 - ☐ Cell
 - ☒ Factory
 - ☒ Store neighbors
 - ☒ Triangle
 - ☒ Scaled normal (sim)
 - ☒ Area (sim)
 - ☒ Midpoint (sim)
 - ☐ ...

Figure 4.2: *An example of our to-do list in onenote*

Over time, we found that relying on the to-do lists and verbal updates became the most efficient approach for this small project. While the GitBoard initially provided a useful overview and helped with orientation, it eventually became more time-consuming than beneficial.

5 Results

The simulation results indicate that the some of the oil will eventually reach the fishing grounds. In the pictures below the initial and a later position of the oil are shown. The red square is marking the fishing ground.

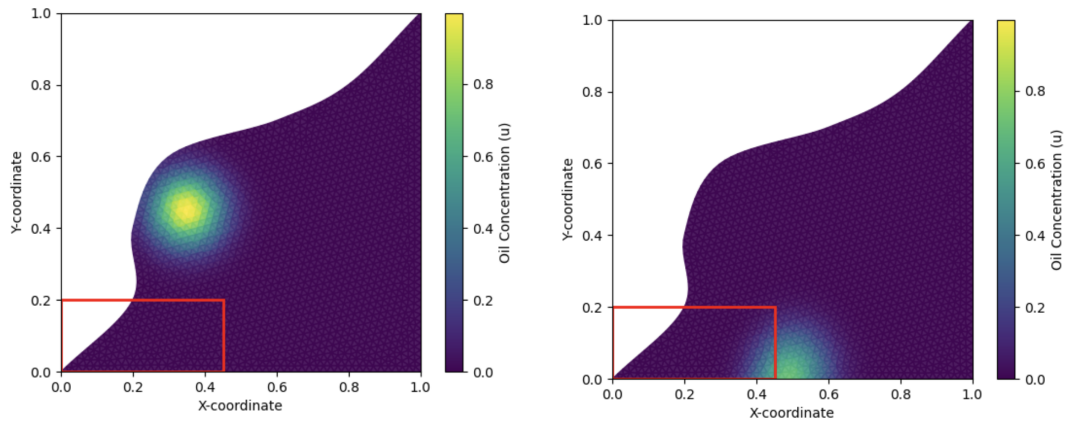


Figure 5.1: *Oil distribution in to the fishing ground*

We experimented with various parameters in our simulation. To obtain a complete simulation, we had to set the total simulation time to 1.5. For example, when we set the timestep to a very small value, the resulting Δt became too large, causing the oil to spread across nearly all the cells. The incorrect simulation outcome is shown in the image below on the left.

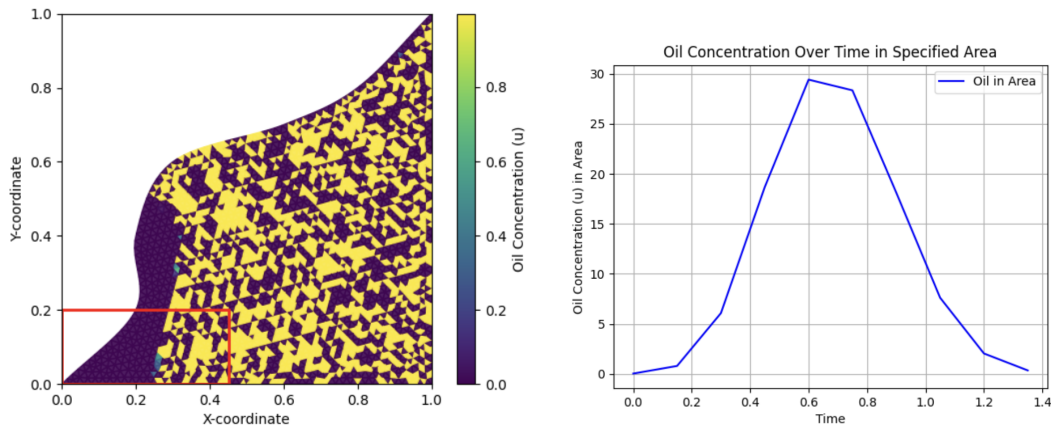


Figure 5.2: *Oil amount with big Δt . Max oil amount in fishing ground*

Additionally, the plot above to the right illustrates the total amount of oil that are passing through the fishing grounds at each time step. The maximum amount of oil that enters the fishing grounds is: 30.

6 Reflections

It has been a fun and exiting time working with this project. Some things we would like to change and make better are commented in this chapter:

The toml config file has an variabel, writefrequency. This variabel's function is to tell how often the mesh should be plotted. So if you have 50 timesteps and 10 in writefrequency, it should be 5 plots. Our understanding of the variabel, was that it should be plotted the same amount of plots as the value of the variabel. So if you have 50 timesteps and 10 in writefrequency, our code plots 10 plots for the 50 timesteps. This was detected at the last minute and should actually be a minor fix, but the time was too short. The task was hard to grasp in the start. It would have been helpful with seeing how the process for each cell mathematically by hand, to really grasp the task and downfalls early.

The reconstruct_mesh function is not working as we wanted. The problem is the way the data is stored as strings. As some of the strings include np.array. When using the eval function that writes the strings into lists and dictionaries, the eval function struggles understanding the np.array calls. This would have been fixed correcting this issue when storing, but there was no time left.